

Revision of class and object in Python

05 August 2026 06:35

In Python, a class is a **blueprint** for creating objects. While an object is a specific instance of that class with its own data. Classes define attributes and behaviors, and objects bring them to life with actual values.

Concept of Class

- A **class** is defined using the **class** keyword
- It groups **attribute** (data) and **methods** (functions).
- Think of a class as a **blueprint** (like a Car design).

```
class Dog:
    species = "Canine" # Class attribute (shared by all dogs)

    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age    # Instance attribute

    def barks(self):
        print(f"{self.name} says Woof!")
```

Concept of Object

- An object is a concrete instance created from a class.
- Each object has its own unique data, but shares the class's structure.
- Think of an object as a **real car built from the blueprint**.

```
class Dog:
    species = "Canine" # Class attribute (shared by all dogs)

    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age    # Instance attribute

    def barks(self):
        print(f"{self.name} says Woof!")
```

```
dog1 = Dog("Buddy", 3)
dog2 = Dog("Lucy", 5)
print(f"Name: {dog1.name} and Age: {dog1.age}")
```

```
print(f"Name: {dog2.name} and Age: {dog2.age}")
dog1.barks() # Buddy says woof!
dog2.barks() # Lucy says woof!
```

Output:

```
Name: Buddy and Age: 3
Name: Lucy and Age: 5
Buddy says Woof!
Lucy says Woof!
```

Difference between class and Object

Class	Object
Blueprint / template.	Actual instance.
Defines attributes and methods.	Holds real data.
Created once, used many times.	Created multiple times.
Example: class Dog	Example: dog1 = Dog("Buddy", 3)

Special Methods

Python classes often use **dunder(double underscore) methods**:

- **__init__(self)**: Constructor, runs when object is created.
- **__str__(self)**: Defines string representation.
- **__len__(self)**: Define behavior for len().

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        return f"{self.title} has {self.pages} pages"

bk1 = Book("Python 3", 365)
bk2 = Book("C++ 26", 665)
bk3 = Book("Java 26", 565)
bk4 = Book("Ruby", 465)
bk5 = Book("Go", 300)
print(bk1)
print(bk2)
```

```
print(bk3)
print(bk4)
print(bk5)
```

Output:

```
Python 3 has 365 pages
C++ 26 has 665 pages
Java 26 has 565 pages
Ruby has 465 pages
Go has 300 pages
```

Analogy:

- **Class** = Recipe(It defines ingredients and steps)
- **Object** = Cake(Actual product made from recipe)
- Multiple cake (objects) can be baked from the same recipe (class) each with different flavors (attributes).

Library Management System

Class Design

Book

- **Attributes:** book_id, title, author, available_copies
- **Methods:** borrow(), returnOfBooks(), displayInfo()

Member

- **Attributes:** member_id, name, email, borrowed_books
- **Methods:** borrow_book(), return_book(), display_borrowed_Books()

```
# Library Management System
```

```
# Class Design
```

```
# Book
```

```
# Attributes: book_id, title, author, available_copies
```

```
# Methods: borrow(), returnOfBooks(), displayInfo()
```

```
# Member
```

```
# Attributes: member_id, name, email, borrowed_books
```

```
# Methods: borrow_book(), return_book(), display_borrowed_Books()
```

```
from datetime import datetime, timedelta
```

```
class Book:
```

```

def __init__(self, book_id, title, author, available_copies):
    self.book_id=book_id
    self.title=title
    self.author=author
    self.available_copies=available_copies

def borrow(self):
    if self.available_copies>0:
        self.available_copies-=1
        return True
    else:
        print(f"Sorry, '{self.title}' is not available.")
        return False

def return_books(self):
    self.available_copies+=1

def __str__(self):
    return f"{self.title} by {self.author} (available: {self.available_copies})"

class Member:

    def __init__(self, member_id, name, email):
        self.member_id=member_id
        self.name=name
        self.email=email
        self.borrowed_books={}

    def borrow_book(self, book):
        if book.borrow():
            due_date= datetime.now()+timedelta(days=14)
            self.borrowed_books[book.book_id]=due_date
            print(f"{self.name} borrowed '{book.title}'. Due on {due_date.date()}")

    def return_book(self, book):
        if book.book_id in self.borrowed_books:
            book.return_books()
            del self.borrowed_books[book.book_id]
            print(f"{self.name} returned '{book.title}'.")
        else:
            print(f"{self.name} did not borrow the book '{book.title}'.")

```

```

def display_borrowed_books(self):
    if not self.borrowed_books:
        print(f"{self.name} has no borrowed books.")
    else:
        print(f"{self.name}'s borrowed books are:")
        for book_id,due_date in self.borrowed_books.items():
            print(f"Book ID: {book_id}, due date: {due_date.date()}")

book1=Book("B001",'Python Basics','Frank',5)
book2=Book("B002",'Python Advanced', 'Horstman',3)

member1=Member(1,'Vaanya', 'vaanya@sharma.com')

print(book1)
print(book2)

member1.borrow_book(book1)
print(book1)
member1.borrow_book(book2)
print(book2)
member1.display_borrowed_books()
member1.return_book(book1)
print(book1)

```

Features Implemented

- Borrowing Decreases Available copies.
- Returning increases available copies.
- Each borrowed book has a due date (14 days)
- Member can track their borrowed books.

Possible Extensions

- **Overdue tracking:** Automatically check if books are overdue.
- **Fine Calculation:** Add penalties for late returns.
- **Search system:** Find books by title or author.
- **Admin Features:** Add / remove books, manage members.

Write a menu driven console application where users can choose options like borrow, return, search, exit interactively.